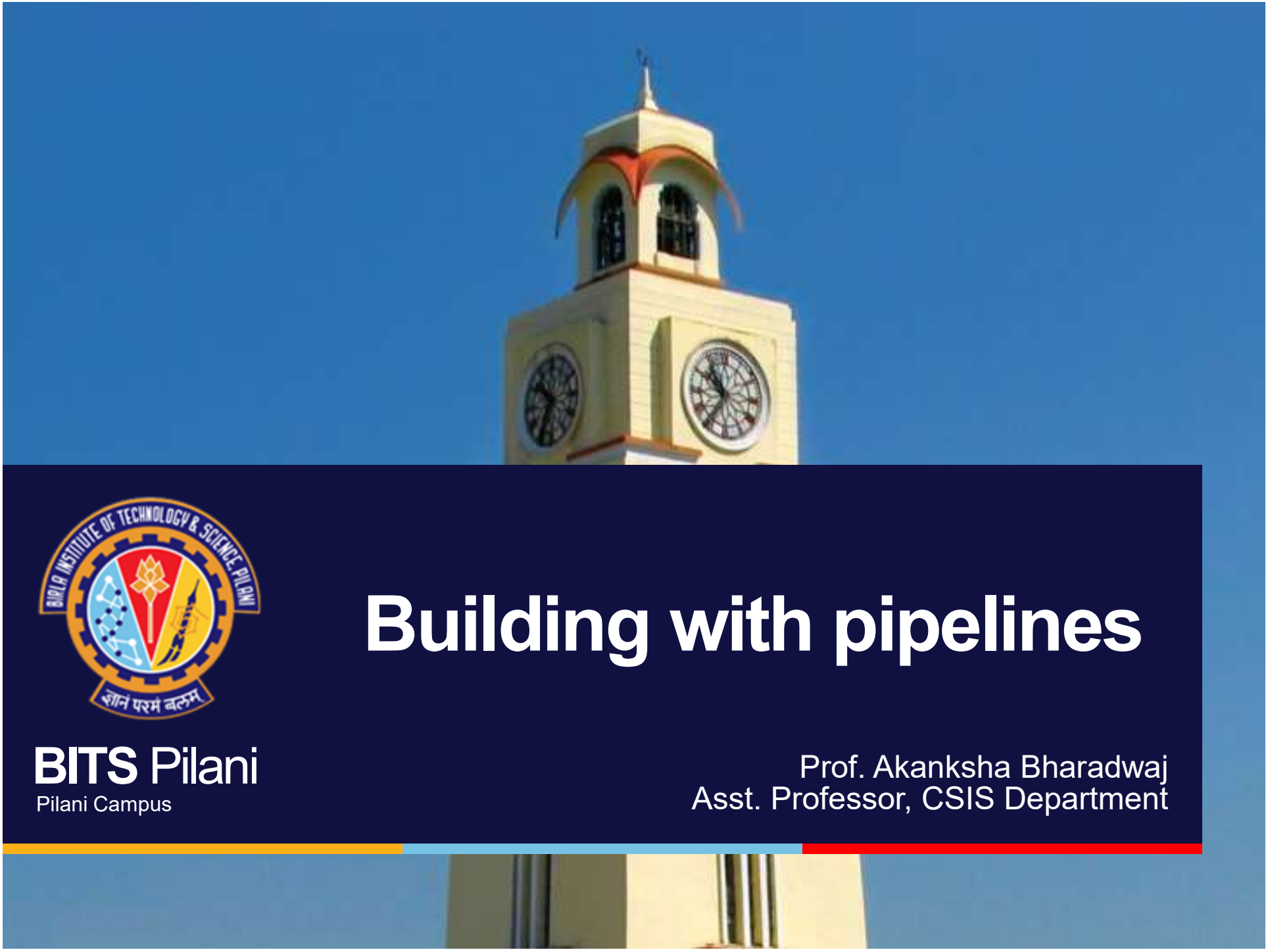


Building with pipelines

BITS Pilani
Pilani Campus

Prof. Akanksha Bharadwaj
Asst. Professor, CSIS Department

innovate achieve lead

BITS Pilani
Pilani Campus

SE ZG583, Scalable Services
Lecture No. 9

Agenda

Building with pipelines

- Continuous integration
- Tooling
- Repository patterns – Multi-repo, mono-repo

DevOps

- DevOps is a **software development methodology** that integrates **development (Dev)** and **IT operations (Ops)** to improve collaboration, automate processes, and accelerate software delivery.

Key Principles of DevOps

- Collaboration & Communication
- Automation
- Continuous Integration (CI)
- Continuous Delivery (CD)
- Monitoring & Feedback
- Security (DevSecOps)

Continuous Integration

- Continuous integration is a DevOps software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run.

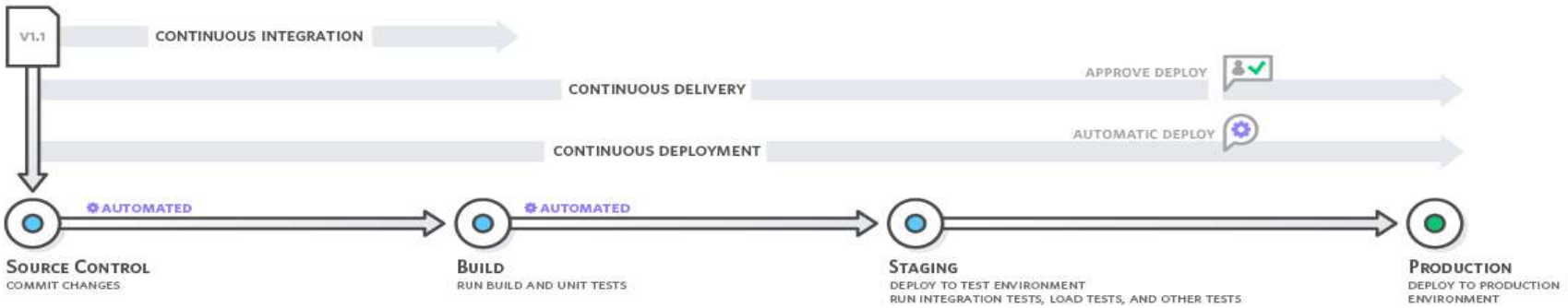
- As part of this process, we often create artifact(s) that are used for further validation, such as deploying a running service to run tests against it.

- To enable the artifacts to be reused, we place them in a repository of some sort, either provided by the CI tool itself or on a separate system.

CI Workflow

innovate achieve lead

- Code Commit
- Automated Build
- Automated Testing
- Feedback and Notifications
- Integration into Main Branch



Benefits of Continuous Integration

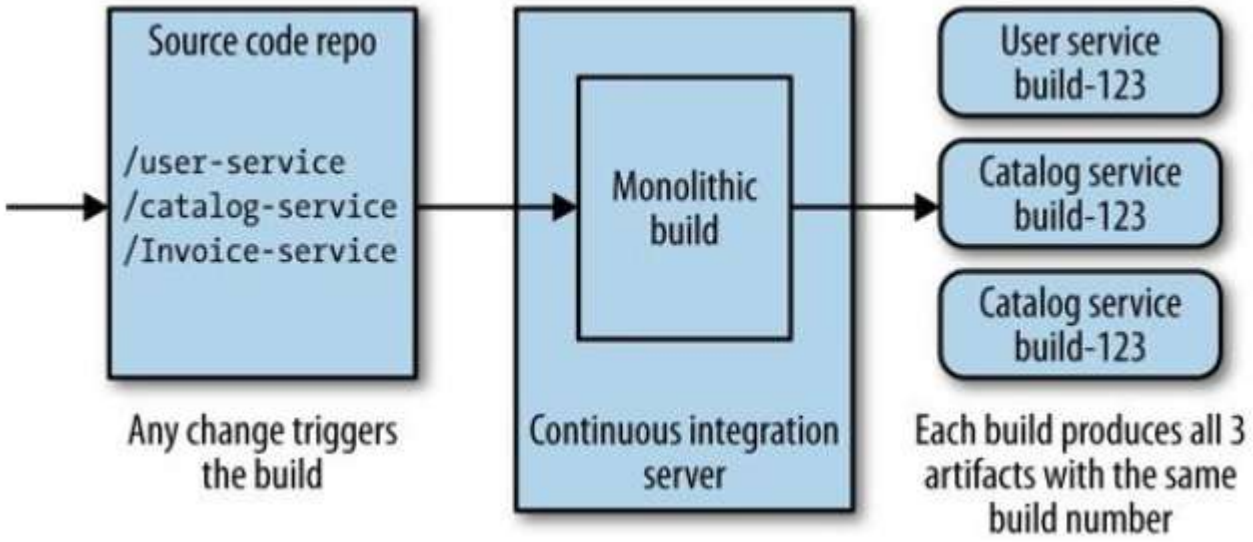
- Early Bug Detection
- Faster Release Cycles
- Improved Code Quality
- Reduced Integration Risks
- Efficient Collaboration

Continuous Integration for Microservices

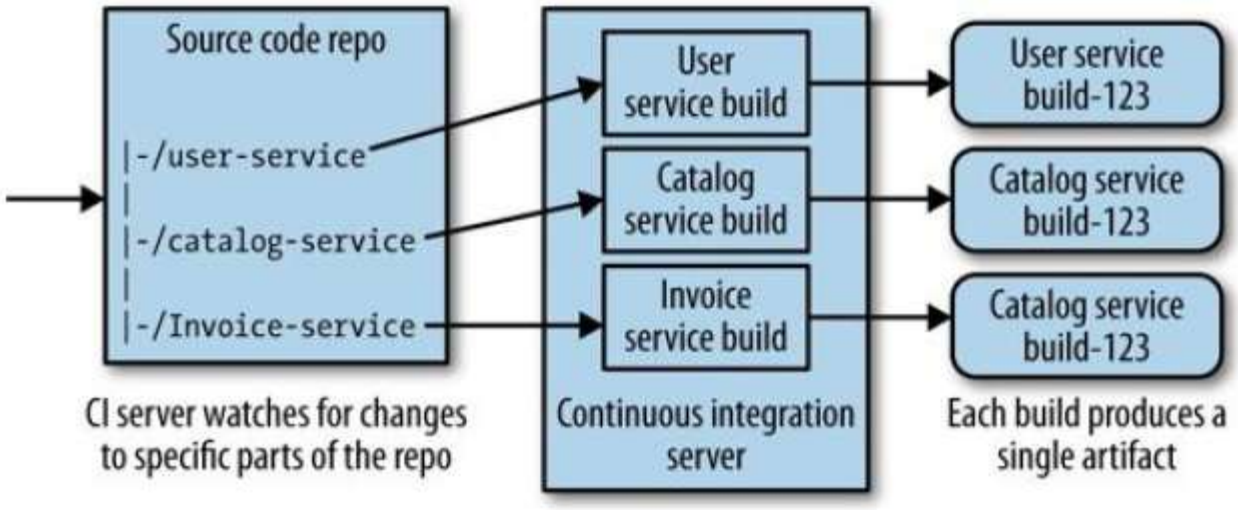
- Continuous Integration (CI) in microservices ensures that each service is built, tested, and integrated frequently while maintaining **independence and scalability**.

Mapping Continuous Integration to Microservices

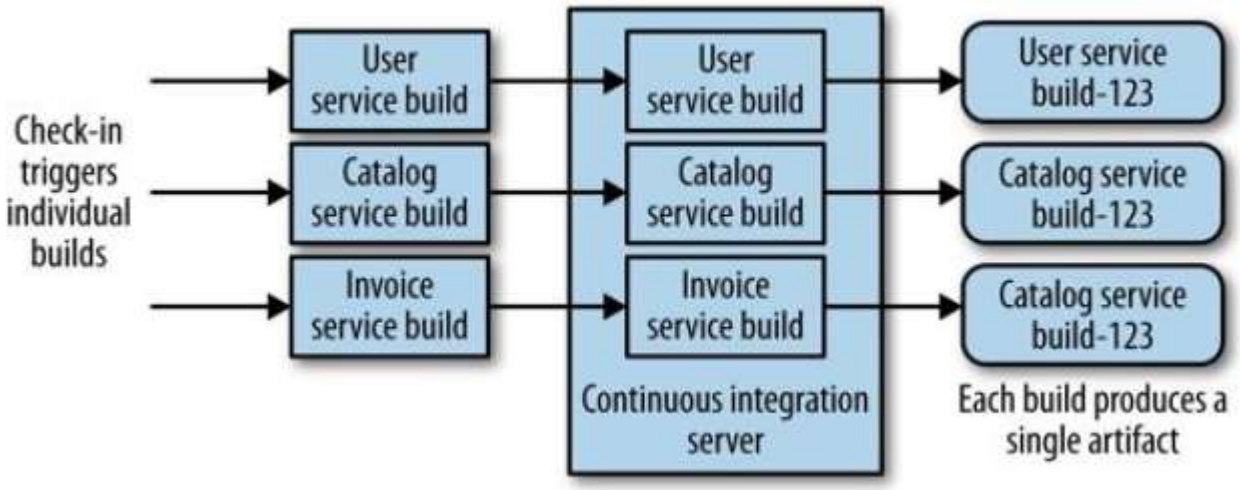
- The simplest option is we could lump everything in together. We will have a single, giant repository storing all our code, and have one single build.



- A variation of previous approach is to have one single source tree with all of the code in it, with multiple CI builds mapping to parts of this source tree



- Best approach is to have a single CI build per microservice, to allow us to quickly make and validate a change prior to deployment into production



Key Considerations for CI in Microservices

- Independent Pipelines
- API Contracts & Testing
- Parallel Testing
- Dependency Management

Continuous Delivery in Microservices

- Continuous Delivery (CD) is the practice of **automating the deployment process** so that software can be released at any time with minimal manual intervention.

- In a **microservices** architecture, each service is independently built, tested, and deployed, enabling faster and more flexible software delivery.

CD Pipeline for Microservices

A typical **Continuous Delivery pipeline** for microservices consists of:

- Continuous Integration (CI)
- Artifact Storage
- Deployment to Staging
- Automated Deployment Strategies
- Monitoring & Feedback Loop

Best Practices for Continuous Delivery in Microservices

- Independent Deployments
- Versioning & Rollbacks
- Deployment Automation
- Monitoring & Logging

Continuous Deployment in Microservices

- Continuous Deployment takes automation a step further by automatically deploying every code change that passes automated tests directly to the production environment without manual intervention.

Deployment Strategies in Microservices

- Rolling Update: Gradually replaces old instances with new ones.
- Blue-Green: Deploys to a parallel environment, then switches traffic.
- Canary Release: Releases to a small % of users before full rollout.
- Feature Flags: Enables/disables features dynamically without redeploying.

Pipeline

innovate achieve lead

- Continuous delivery is the approach whereby we get constant feedback on the production readiness of each and every check-in, and furthermore treat each and every check-in as a release candidate

Figure 6-4. A standard release process modeled as a build pipeline

- In a microservices world, where we want to ensure we can release our services independently of each other, it follows that as with CI, we'll want one pipeline per service.

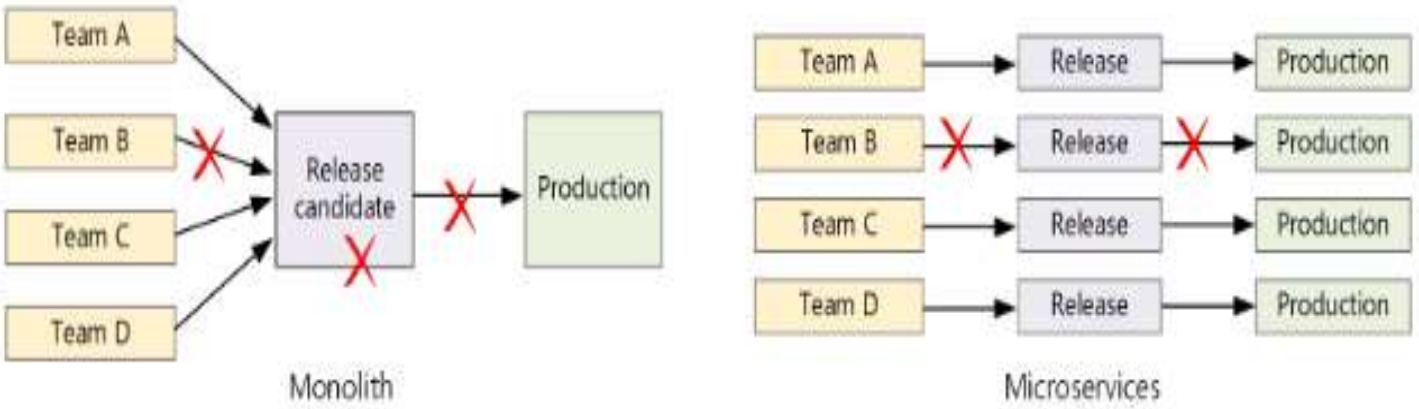
CI/CD pipeline for Monolith Vs Microservices

Monolithic

- Single build pipeline
- Bug fixes impacts release

Microservice

- One pipeline per service
- Issue in one service doesn't impact the other service



Platform-Specific Artifacts

- From the point of view of a microservice, depending on your technology stack, the artifact may not be enough by itself.
- While a Java JAR file can be made to be executable and run an embedded HTTP process, for things like Ruby and Python applications, you'll expect to use a process manager running inside Apache or Nginx.
- So we may need some way of installing and configuring other software that we need in order to deploy and launch our artifacts.
- This is where **automated configuration management** tools like Puppet and Chef can help

Service Configuration

- If we have some configuration for our service that does change from one environment to another, how should we handle this as part of our deployment process?
- One option is to build one artifact per environment, with configuration inside the artifact itself.
- A better approach is to create one single artifact, and manage configuration separately.

Repository Strategies for Microservices

- Managing code repositories in a **microservices architecture** is crucial for efficient development, version control, and deployment.
- Types of Repository Structures for Microservices:
  - Monorepo (Single Repository for All Microservices): A **single Git repository** containing code for all microservices.
  - Multirepo (One Repository per Microservice): Each microservice has its **own separate Git repository**.

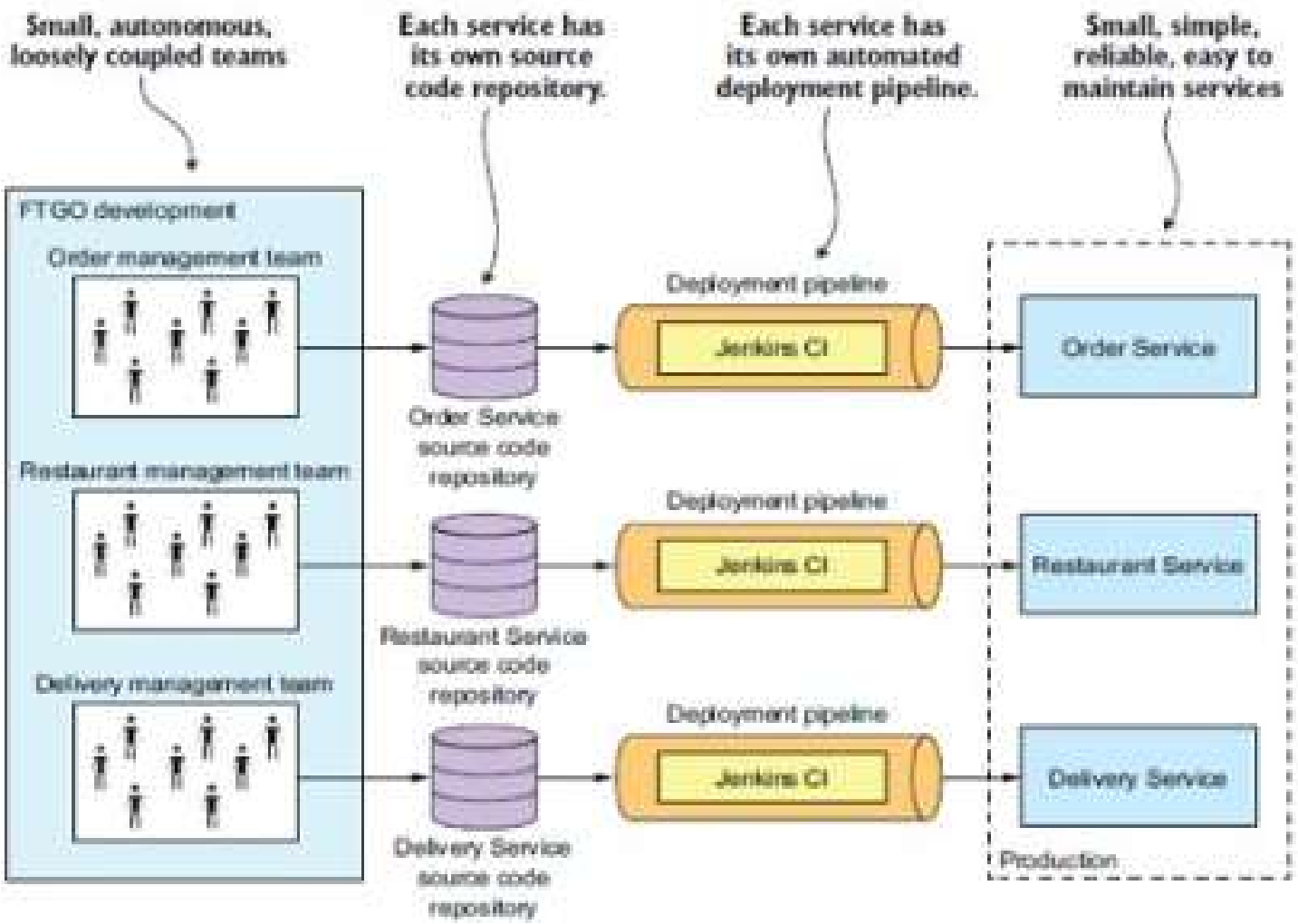
Mono-repo vs. Multi-repo

	Monorepo	Multiple repos
Advantages	Code sharing Easier to standardize code and tooling Easier to refactor code Discoverability - single view of the code	Clear ownership per team Potentially fewer merge conflicts Helps to enforce decoupling of microservices
Challenges	Changes to shared code can affect multiple microservices Greater potential for merge conflicts Tooling must scale to a large code base Access control More complex deployment process	Harder to share code Harder to enforce coding standards Dependency management Diffuse code base, poor discoverability Lack of shared infrastructure

Deployment Challenges

- Many small independent code bases.
- Multiple languages and frameworks
- Release management
- Service updates
- Integration and load testing

Pipeline for FTGO application



Self Study

https://thenewstack.io/your-ci-cd-pipeline-wasnt-built-for-microservices/

References

- Books as per the handout
- https://aws.amazon.com/devops/continuous-integration/
- https://docs.microsoft.com/en-us/azure/architecture/microservices/ci-cd
- https://medium.com/@dmosyan/ci-cd-for-microservices-1b5582f3e1fd